

Semantic Clustering of Resume Data Using Transformer Embeddings, UMAP and HDBSCAN

By

Rshijuta Pokharel

UID : U01122370

Presented to:

The Faculty of Data Science

Department to Computer Science and Engineering

Russ Engineering College

Wright State University

For the Degree of Master's in Science, Data Science

Under the Supervision of:

Dr. Wen Zhang

Dayton, Ohio

April 28, 2026

Table of Contents

<i>ABSTRACT</i>	4
1. <i>Introduction</i>	5
2. <i>Literature Review</i>	5
3. <i>Problem Statement</i>	6
4. <i>Dataset</i>	7
5. <i>Pipeline Flow Diagram</i>	7
6. <i>Methodology</i>	8
6.1. <i>Text Extraction and Preprocessing</i>	8
<i>Text Extraction</i>	8
<i>Text Cleaning</i>	8
<i>Text Normalization and Word Segmentation</i>	9
6.2. <i>Embedding Generation</i>	9
<i>Model Overview</i>	9
<i>Model Specifications</i>	10
<i>Chunk-Based Representation</i>	11
<i>Instruction-Based Embedding</i>	11
<i>Anchor-Based Semantic Guidance</i>	12
6.3. <i>Embedding Validation Methods</i>	14
<i>Normalization Check</i>	14
<i>Cosine Similarity Distribution</i>	15
<i>Nearest Neighbor Analysis</i>	15
6.4. <i>Dimensionality Reduction</i>	16
<i>Principal Component Analysis (PCA)</i>	16
<i>Uniform Manifold Approximation and Projection (UMAP)</i>	16
<i>Combined PCA and UMAP</i>	17
6.5. <i>Clustering Using HDBSCAN</i>	18
<i>Overview of HDBSCAN Algorithm</i>	18
<i>Clustering Procedure</i>	18
<i>HDBSCAN Parameters</i>	19
<i>Output and Visualization Methods</i>	19
6.6. <i>Label Preparation for Evaluation</i>	19
6.7. <i>Evaluation Metrics</i>	20
<i>Internal Evaluation</i>	20

<i>External Evaluation</i>	21
<i>Semantic Validation</i>	22
7. <i>RESULTS & DISCUSSION</i>	23
7.1. <i>Embedding Validation Output</i>	23
7.2. <i>Clustering Performance Overview</i>	25
7.3. <i>Effect of Dimensionality Reduction</i>	29
7.4. <i>Evaluation Results</i>	29
7.5. <i>Semantic Validation Results</i>	31
7.6. <i>Summary of Observations</i>	31
7.7. <i>Limitations</i>	31
8. <i>Conclusion</i>	32
<i>REFERENCES</i>	33

ABSTRACT

This project focuses on clustering technical resumes based on their semantic similarity using an unsupervised learning approach. Resume data is unstructured and often contains overlapping information such as skills and experiences, which makes it difficult to group into clearly distinct categories. In this project, resumes are first converted into numerical representations using transformer-based embeddings. Dimensionality reduction techniques like Principal Component Analysis (PCA) and Uniform Manifold Approximation and Projection (UMAP) are then applied before clustering using the HDBSCAN algorithm. Different configurations are compared to evaluate clustering performance. The results show that applying UMAP, especially after PCA, significantly improves clustering quality. However, due to the similarity across resumes, perfect separation is not expected. This study highlights the importance of dimensionality reduction when working with high dimensional text data and shows that meaningful clusters can still be formed despite the complexity of resume information.

Keywords: semantic clustering, resume analysis, transformer embedding, dimensionality reduction, PCA, UMAP, HDBSCAN, unsupervised learning

1. Introduction

Organizing and analyzing resume data is an important task in areas such as recruitment and candidate screening. When dealing with a large number of resumes, it becomes difficult to manually identify similar profiles. Automatically grouping resumes based on their content can help simplify the process and improve efficiency.

However, clustering resume data is challenging. Resumes contain different types of information such as skills, education, and work experience, and many of them share common tools and technologies. This leads to high similarity between documents and makes it difficult to form clearly separated groups using traditional clustering methods.

Recent advances in natural language processing have enabled the use of transformer-based models to represent text as dense vector embeddings. These embeddings capture semantic meaning and allow comparison between documents based on contextual similarity [1]. In this project, a BAAI BGE embedding model is used to generate these representations. Since the resulting embeddings are high-dimensional, clustering them directly is difficult.

To address this, dimensionality reduction techniques and clustering methods are combined. Modern clustering approaches often integrate representation learning with clustering to better handle complex and high-dimensional data [2]. In this work, PCA and UMAP are used to reduce dimensionality before applying HDBSCAN for clustering.

The goal of this project is to compare different representations of the data, including raw embeddings, PCA, UMAP, and PCA combined with UMAP, and evaluate how these affect clustering performance. This helps in understanding how dimensionality reduction influences clustering quality for real-world resume data.

2. Literature Review

Clustering of textual data presents significant challenges due to noise, sparsity and overlapping semantic structures. Transformer-based models such as Sentence-BERT [1] and BGE [3] have improved representation by mapping documents into dense semantic vector spaces, enabling clustering based on semantic relationships.

The Transformer architecture [4] forms the foundation of these models by capturing relationships through self-attention. However, high-dimensional embeddings often require dimensionality reduction to improve cluster structure and computational efficiency. Techniques such as Principal Component Analysis (PCA) [7] and Uniform Manifold Approximation and Projection (UMAP) [5] are widely used for this purpose. UMAP in particular preserves local neighborhood structure, making it effective for clustering tasks.

The deep clustering survey by Min et al. [2] highlights that real-world datasets often contain ambiguous boundaries and overlapping categories, which limits the effectiveness of strict

clustering. This is especially relevant for resume data, where roles such as academic and industry positions may share common skills and experiences. The survey also emphasizes the importance of combining internal and external evaluation metrics, as no single metric fully captures clustering quality.

Recent work such as BERTopic [8] demonstrates an effective pipeline combining transformer embeddings, UMAP and HDBSCAN [6] for clustering textual data. This approach motivates the pipeline used in this project, where embeddings are reduced and clustered using density-based methods. HDBSCAN is particularly suitable as it can automatically determine the number of clusters and identify noise points, which is important for datasets with unclear boundaries.

Existing resume analysis approaches often rely on traditional methods such as TF-IDF and supervised models [9] or complex deep learning pipelines focused on skill extraction and ranking [10]. These approaches differ from this project as they either depend on labeled data or focus on prediction tasks rather than unsupervised semantic grouping.

In contrast, this project performs unsupervised clustering using full document embeddings. However, evaluation remains challenging due to the nature of the dataset. The manually created labels used for evaluation do not represent true ground truth, but rather human interpretations of resume categories. Additionally, overlap between categories and differences between semantic content and label definitions lead to inconsistencies between clustering results and external metrics. As a result, clustering may be semantically meaningful even when alignment with labels appears low.

3. Problem Statement

Clustering resume data based on semantic similarity is a challenging task due to the unstructured nature of resumes and the significant overlap in content across different profiles. Resumes often contain common skills, tools and experiences which makes it difficult to form clearly separable clusters

Traditional text representation methods such as TF-IDF fail to capture contextual meaning, leading to poor grouping of semantically similar documents. While transformer-based embeddings provide richer semantic representations, they result in high-dimensional feature spaces that are not directly suitable for clustering.

Additionally, real-world resume data does not have well-defined or mutually exclusive categories. Roles such as early-career and mid-level professionals often overlap. The manual labels may not accurately reflect underlying semantic relationships. This creates further difficulty in evaluating clustering performance using external metrics.

Therefore, there is a need for an approach that captures semantic relationships between resumes, reduces high-dimensional embedding space effectively, supports clustering without requiring predefined categories and enables meaningful evaluation.

This project addresses these challenges by combining transformer-based dimensionality reduction techniques and density-based clustering methods to group resumes based on their semantic similarity.

4. Dataset

The dataset used in this study consists of 73 real resume documents in PDF format. Each file contains the full content of an individual resume, including sections such as education, experience, skills and publications. The resumes primarily belong to technical and academic domains, resulting in a high degree of similarity across documents.

The different categories of the resumes based on manual observation is enlisted as follows:

5. Pipeline Flow Diagram

The overall pipeline begins with raw resume documents, which are processed through text extraction and preprocessing steps to obtain clean textual data. The text is then divided into chunks and converted into semantic embeddings using a transformer-based BGE model with instruction-based input. Relevant chunks are filtered using anchor-based similarity scoring and aggregated into a single embedding per resume.

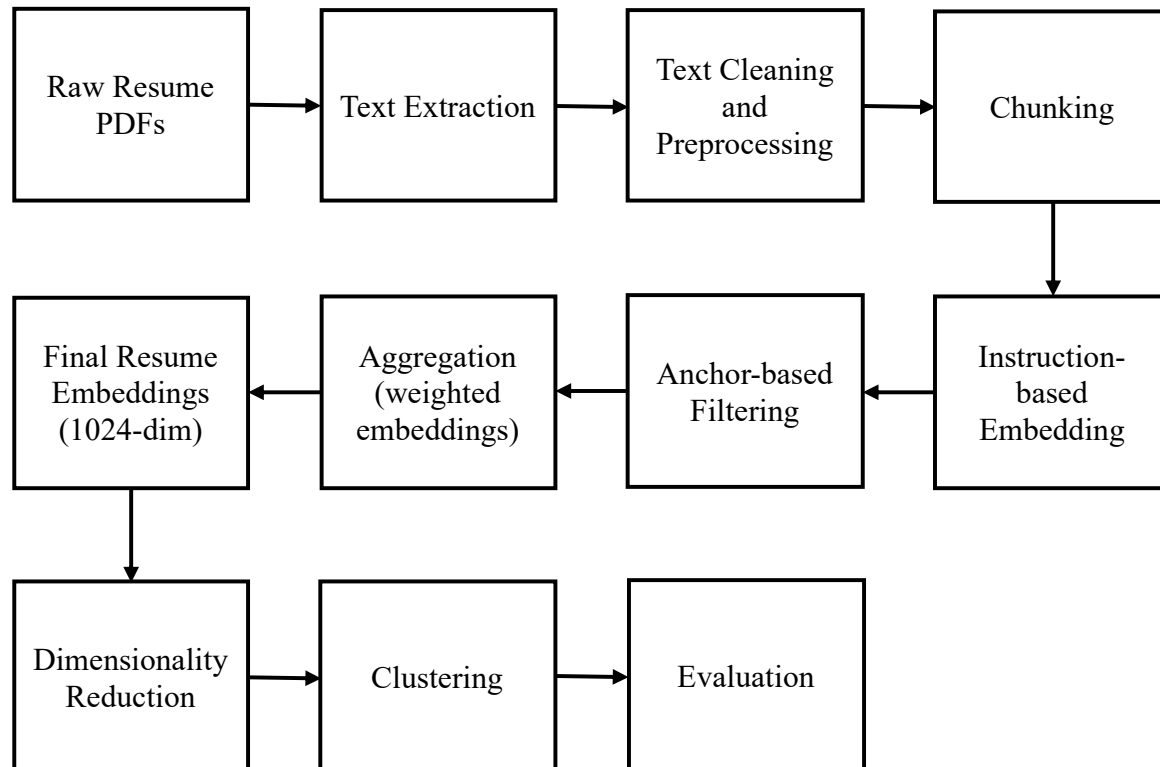


Figure 5.1: Overview of the Resume Clustering Pipeline

These high-dimensional embeddings are reduced using PCA and UMAP to improve clustering structure. The reduced representations are then clustered using HDBSCAN which identifies dense groups and noise points. Finally, clustering performance is evaluated using internal, external and semantic validation metrics.

6. Methodology

6.1. Text Extraction and Preprocessing

The dataset used in this study consists of resume documents in PDF format. Since resumes are semi-structured and contain a mixture of textual content, formatting elements and non-informative metadata, an initial preprocessing stage is essential to convert the raw data into a consistent and analyzable textual representation.

Text Extraction

Text extraction is performed using the *pdfplumber* library, which enables structured reading of PDF content. Each resume is processed page-by-page, using parameters like *x-tolerance* and *y-tolerance* which control how closely characters must align spatially to be grouped into words. This improves extraction quality for formatted documents.

Text Cleaning

The extracted text is processed line-by-line to preserve the structural layout of the resume (sections like education and experience). The cleaning process is implemented using Python string operations, regular expressions (*re* module) and rule-based transformations to systematically remove noise and standardize the text. The following types of noise are eliminated:

- Email addresses, URLs, phone numbers: They are removed using regular expression pattern matching (e.g. patterns matching *text@domain*, *http/https* links and numeric phone formats). These elements are considered non-informative for semantic clustering.
- Timestamps, dates and year information: They are removed by detecting common patterns such as time formats (e.g. 10:30 AM), year values (e.g. 2022) and month-year combinations. This prevents temporal information from influencing similarity between resumes.
- Unnecessary special characters and formatting artifacts: They are removed or corrected through string normalization, including fixing spacing issues and eliminating redundant symbols introduced during PDF extraction.
- Personal names in the first two lines: They are removed using pattern-based matching of capitalized word pairs. This step reduces bias, ensuring clustering is driven by skills and experience rather than identity.

These operations are applied sequentially to each line, followed by whitespace normalization to produce clean and consistent text.

Text Normalization and Word Segmentation

After noise removal, additional normalization steps are applied to ensure consistency in the text. These are implemented using Python string operations and regular expressions, where extra whitespace is removed using string splitting and joining, spacing issues are corrected through pattern-based substitutions and improperly joined words are separated.

To handle merged words resulting from PDF extraction, the *wordninja* library is used for word segmentation. In our case, only words with length greater than 10 characters are considered for splitting. Words containing special characters such as hyphens (-) or periods (.) are excluded. The split is accepted only if the segmentation results in more than one meaningful word. This controlled approach ensures that only likely merged words are corrected, while avoiding incorrect splitting of valid tokens.

Additionally, CamelCase words are separated using regular expression rules and missing spaces after punctuation are corrected to improve readability and consistency.

After preprocessing, the cleaned and normalized text for each resume is stored as a separate “.txt” file in the “data/extracted_text” directory. Each file corresponds to a single resume, preserving the mapping between the original PDF documents and their processed textual representations. This structured output serves as the input for the embedding generation stage.

6.2. Embedding Generation

To represent resumes numerically, transformer-based embedding models are used. In this work, a BAAI BGE embedding model is applied through the SentenceTransformers framework to generate semantic representations of resume text.

Model Overview

The BGE (Bidirectional General Embedding) model is a transformer-based sentence embedding model designed to map text into dense vector representations [3]. It is built on the Transformer architecture, where contextual relationships between words are captured using a self-attention mechanism [4].

Given an input sequence of tokens $x = \{x_1, x_2, \dots, x_n\}$, the model first encodes it into fixed-length vector representation:

$$E = f_{\theta}(x), \quad E \in R^d$$

where:

- f_θ represents the embedding model
- $d = 1024$ embedding dimensions

The resulting embedding captures the overall semantic meaning of the input text, allowing comparison between different documents in a shared vector space.

According to model documentation [3], BGE is trained using contrastive learning where pairs of semantically similar texts are mapped closer together in the embedding space, while dissimilar pairs are pushed farther apart. This is typically achieved using large-scale text pair datasets and optimized using similarity-based loss functions. As a result, cosine similarity between embeddings serves as an effective measure of semantic relatedness.

A key feature of BGE models is their support for instruction-based embedding, where a task-specific instruction is prepended to the input text. This allows the model to generate embeddings that are tailored to a specific downstream task, such as clustering or retrieval. In this work, instruction-based inputs are used to guide the model toward capturing relevant resume characteristics, including career stage, academic role and professional orientation.

The model internally processes the input using multiple transformer layers to produce contextualized token representations. Within each layer, self-attention is applied, allowing each token to attend to all other tokens in the sequence. The tokens are then aggregated into a single embedding vector. This aggregation ensures that the final representation captures both local and global semantic information from the input text.

Due to its training method and design, the BGE model produces embeddings that are highly effective for semantic similarity tasks. This makes it particularly suitable for clustering applications, where the goal is to group resumes based on their underlying semantic content rather than surface-level textual features.

Model Specifications

The model used in this work has the following characteristics:

- Model Name: BAAI/bge-large-en-v1.5
- Embedding Dimension: 1024
- Framework: SentenceTransformers
- Similarity Metric: Cosine similarity
- Normalization: Enabled (normalized_embeddings = True)

These specifications ensure high-quality semantic representations suitable for clustering tasks.

BAAI/bge-large-en-v1.5 Model:

This model is a large-scale English embedding model developed by the Beijing Academy of Artificial Intelligence (BAAI) [3]. It is a part of BGE family and is optimized for generating high-

quality semantic embeddings for tasks such as retrieval, clustering and similarity search. The large variant provides higher representational capacity compared to smaller models, enabling it to capture more complex semantic relationships in texts. The model has an embedding dimension of 1024 and is trained on large-scale text pair datasets. The v1.5 version includes improvements in embedding quality and stability, particularly for instruction-based inputs.

SentenceTransformers Framework:

The embedding model is implemented using the SentenceTransformers framework, which provides an efficient interface for generating sentence and document embeddings using transformer-based architectures [1]. It extends pretrained transformer models (such as BERT) by fine-tuning them using Siamese or triplet network structures to produce semantically meaningful embeddings. The framework also supports direct computation of cosine similarity and embedding normalization.

Chunk-Based Representation

Resumes are typically long documents containing multiple sections such as education, skills and experience. Embedding the entire resume as a single block can lead to a loss of important information as the model compresses all content into a single vector and could overlook section-level details.

Additionally, transformer-based models such as BAAI/bge-large-en-v1.5 have a maximum input length of approximately 512 tokens. This limitation prevents the model from processing long documents in a single pass, making it necessary to divide the text into smaller segments.

To address these issues, each resume is divided into smaller segments (chunks), allowing the model to process the entire document while capturing localized semantic information. A resume text T is represented as:

$$T = \{C_1, C_2, \dots, C_n\}$$

where C_i denotes the i -th chunk.

In this project, chunking is implemented using word-based approach. The text is split into consecutive segments of approximately 500 words per chunk, without overlap. The final chunk may contain fewer words depending on the document length. This approach ensures that the input length constraint of the model is respected, all parts of the resume are processed and section-level semantic information is preserved.

Instruction-Based Embedding

To improve the quality and relevance of embeddings, an instruction-based embedding strategy is used. Instead of encoding raw text directly, each chunk is augmented with a task-specific instruction that guides the model toward capturing features relevant to resume clustering.

For this project, the instruction used is:

INSTRUCTION = "Represent this resume for clustering based on career stage, academic rank, research vs industry orientation, and professional role."

For each chunk C_i , the instruction I is concatenated with the text before embedding:

$$x_i = I + C_i$$

The resulting input x_i is then passed to the embedding model to generate a vector representation:

$$E_i = f_{\theta}(x_i)$$

where:

- f_{θ} represents BGE model
- $E \in R^{1024}$ is the embedding of chunk i

This approach ensures that the generated embeddings are aligned with the specific objective of clustering resumes based on professional characteristics.

Anchor-Based Semantic Guidance

In addition to instruction-based input, a set of predefined anchor descriptions is used to further guide the embedding process. This approach is inspired by semantic similarity and retrieval-based filtering techniques, where predefined reference descriptions are used to identify relevant content [1], [3].

These anchors represent common resume profiles and help identify semantically relevant content with each document. They act as reference points in the embedding space capturing different types of professional roles. By comparing each chunk of a resume to these anchors, the model can determine which parts of the text are most relevant to meaningful career-related categories.

The anchors used in this project are:

ANCHORS = [
"senior academic professor tenured faculty research leadership publications grants",
"assistant professor early career academic teaching research experience post phd",
"research scientist postdoctoral researcher academic research lab publications",
"industry research leader principal scientist applied research corporate leadership",
"industry professional software engineer data scientist applied industry experience",
"teaching focused academic lecturer instructor curriculum teaching university",
"mid level academic associate professor research teaching balance",
"hybrid academic industry collaboration startup entrepreneur research commercialization"
]

Each anchor A_j is also embedded using the same instruction-based format:

$$A_j' = f_{\theta}(I + A_j)$$

As a result, this ensures that embedding representation emphasizes important resume characteristics rather than uniformly treating all sections of the text.

Relevance Scoring:

To determine the importance of each chunk, cosine similarity is computed between chunk embeddings and anchor embeddings. Cosine similarity measures the similarity between two vectors based on the angle between them.

For each chunk embedding E_i and anchor embedding A_j' , the similarity is defined as:

$$\text{sim}(E_i, A_j') = \frac{E_i \cdot A_j'}{\|E_i\|_2 \cdot \|A_j'\|_2}$$

where:

- E_i represents the embedding of the i -th chunk
- A_j' represents the embedding of the j -th anchor

For each chunk, the maximum similarity score across all anchors is selected”

$$s_i = \max_j \text{sim}(E_i, A_j')$$

The score s_i represents the semantic relevance of the chunk with respect to the predefined anchor profiles. Chunks with higher similarity scores are considered more relevant, as they align more closely with meaningful professional categories.

Filtering Strategy:

Based on relevance scores s_i computed for each chunk, a filtering step is applied to retain only the most informative portions of the resumes. Specifically, chunks are ranked in descending order of their similarity scores, and only the top 70% of the chunks are selected:

$$k = \max(1, \lfloor 0.7 \cdot n \rfloor)$$

where:

- n is the total number of chunks
- k is the number of retained chunks

This filtering removes less relevant or generic content and ensures that the final representation focuses on semantically meaningful information aligned with anchor profiles.

Weighted Aggregation:

The selected chunk embeddings are combined into a single resume-level representation using a weighted aggregation approach. Each chunk is assigned a weight based on its own relevance score and these weights are normalized using a softmax function, which converts a set of raw scores into normalized weights that sum to 1.

$$w_i = \frac{e^{s_i}}{\sum_{j=1}^k e^{s_j}}$$

where:

- s_i is the similarity score of the i -th chunk
- s_j is the similarity score of all selected chunks

The final embedding is computed as:

$$E_{\text{final}} = \sum_{i=1}^k w_i \cdot E_i$$

where:

- E_i is the embedding of the i -th selected chunk
- w_i is the normalized weight

This combination of filtering and weight aggregation ensures that more relevant chunks contribute more strongly to the final embedding and irrelevant or noisy content is minimized. The resulting representation captures the most important semantic aspects of each resume.

The final embeddings are stored as NumPy array of shape (73 x 1024) in the file “*resume_embeddings.npy*”, along with corresponding filenames in a *filenames.txt* file to maintain mapping between embeddings and original resumes.

6.3. Embedding Validation Methods

Before applying dimensionality reduction and clustering, the quality of generated embeddings is evaluated to ensure that they provide meaningful semantic representations. The validation focuses on normalization, similarity structure and neighborhood consistency.

Normalization Check

To verify that embeddings are properly scaled, the L2 norm of each embedding vector is computed. The L2 norm measures the magnitude of an embedding vector which is computed as:

$$\|E_i\|_2 = \sqrt{\sum_{k=1}^d E_{ik}^2}$$

where:

- d = dimension of embedding

- k = index of each component in the vector
- E_{ik} is k -th value inside the i -th embedding

Since embeddings are generated with normalization enabled, the norm of each vector is expected to be close to 1. Consistent norm values indicate that embeddings are properly normalized and suitable for cosine-similarity based comparisons.

Significant deviations in norm values may indicate issues in embedding generation, while extremely low norms may suggest incomplete or invalid embeddings.

Cosine Similarity Distribution

To analyze the semantic structure of embedding space, pairwise cosine similarity is computed between all embeddings where E_i and E_j represent embedding of two different resumes:

$$\text{sim}(E_i, E_j) = \frac{E_i \cdot E_j}{\|E_i\|_2 \cdot \|E_j\|_2}$$

Since embeddings are L2-normalized, cosine similarity reduces to a dot product. The resulting similarity values lie in the range:

$$-1 \leq \text{sim}(E_i, E_j) \leq 1$$

The value 1 indicates maximum similarity (identical semantic direction) and 0 indicates no similarity (orthogonal vectors).

To evaluate the overall structure, the distribution of similarity values is analyzed using summary statistics and histogram. Moderate similarity values indicate meaningful semantic variation across resumes. Very high similarity values across pairs may indicate embedding collapse where resumes are mapped too closely, while very low similarity values may indicate weak semantic structure or poor representation.

A well-distributed range of similarity values suggests that the embeddings effectively capture differences and similarities between resumes. Cosine similarity is widely used for evaluating semantic similarity in embedding spaces [1].

Nearest Neighbor Analysis

To further validate embedding quality, a nearest neighbor analysis is performed. For each embedding E_i , the most similar embeddings are identified based on cosine similarity:

$$\text{NN}_k(E_i) = \arg \max_{E_j} \text{sim}(E_i, E_j)$$

This step verifies whether semantically similar resumes are positioned close to each other in the embedding space. Consistent grouping of similar profiles indicates that the embeddings preserve meaningful relationships.

In this project, it is performed by identifying the top 5 most similar embeddings for each resume. This corresponds to a k-nearest neighbor search ($k = 5$) in the embedding space. If the nearest neighbors correspond to resumes with similar profiles, it indicates our embeddings are well placed.

6.4. Dimensionality Reduction

High-dimensional embeddings often contain redundant information and can be difficult to cluster effectively due to the curse of dimensionality. To improve the structure of the embedding space and enhance clustering performance, dimensionality reduction techniques are applied.

The two methods used in this project are:

- Principal Component Analysis (PCA)
- Uniform Manifold Approximation and Projection (UMAP)

These methods are applied individually and in combination to evaluate their impact on clustering quality.

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a linear dimensionality reduction technique that projects high-dimensional data onto a lower-dimensional space while preserving maximum variance [7].

Given an embedding matrix $X \in \mathbb{R}^{n \times d}$, PCA computes a set of orthogonal directions (principal components) such that:

$$Z = XW$$

where:

- $W \in \mathbb{R}^{d \times k}$ contains the top k principal components
- $Z \in \mathbb{R}^{n \times k}$ is the reduced representation

In our case, PCA is applied to reduce the 1024-dimensional embeddings to a lower-dimensional space i.e. 50 dimensions. This step removes noise and redundant features while preserving the most significant variance in data.

PCA is used as a preprocessing step to improve computational efficiency and stabilize the embedding space before applying nonlinear dimensionality reduction.

Uniform Manifold Approximation and Projection (UMAP)

UMAP is a non-linear dimensionality reduction technique that preserves both local and global structure of high-dimensional data by modeling its underlying manifold [5]. UMAP constructs a graph representation of the data based on nearest neighbors and optimizes a low-dimensional embedding such that the structure of the graph is preserved. This makes it well-suited for preparing embeddings for clustering algorithms such as HDBSCAN.

UMAP parameters used in our project are:

- `n_neighbors = 10`
It controls the size of the local neighborhood used for the manifold approximation. The smaller values emphasize local structure. In this project, a value of 10 is chosen to balance local and global structure.
- `n_components = 10`
It defines the dimensionality of the output space. A value of 20 balances information retention and computational efficiency. Reducing to very low dimensions may lead to loss of important structures.
- `min_dist = 0.1`
It controls how tightly points are packed in the reduced space. Lower values allow tighter clustering, so the value 0.1 is used in the project to encourage tight grouping of similar resumes. This helps to improve the cluster separability.
- `metric = cosine`
It defines how distances between points are computed. Since embeddings are generated using cosine similarity, the same metric is used in UMAP to ensure consistency between embedding space and dimensionality reduction.

Combined PCA and UMAP

In addition to applying PCA and UMAP individually, a combined dimensionality reduction approach is used, where PCA is applied first, followed by UMAP.

This approach is motivated by the high dimensionality of transformer-based embeddings. PCA is first applied to reduce dimensionality and remove less informative variance, producing a more compact representation. UMAP is then applied to this reduced space to capture nonlinear relationships and preserve local neighborhood structure.

Using PCA as a preprocessing step improves the efficiency of UMAP and helps produce a more stable low-dimensional representation. The resulting embedding space is suited for clustering, as it retains important structural information while reducing noise.

Dimensionality reduction is applied after embedding generation and validation, and before clustering. The reduced embeddings are then used as input to the HDBSCAN clustering algorithm. This step is essential for improving the structure of the data and enabling meaningful cluster formation.

6.5. Clustering Using HDBSCAN

After dimensionality reduction, clustering is performed to group resumes based on semantic similarity. In this project, the HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) algorithm is used to identify clusters in the embedding space.

Overview of HDBSCAN Algorithm

HDBSCAN is a density-based clustering algorithm that extends DBSCAN by building a hierarchy of clusters and selecting the most stable ones [6]. Unlike traditional clustering methods such as k-means, HDBSCAN does not require the number of clusters to be specified in advance.

HDBSCAN clustering is applied to the reduced embedding space to identify groups of semantically similar resumes. Given a dataset $X = \{x_1, x_2, \dots, x_n\}$, the algorithm begins by estimating the local density around each data point. This is achieved by computing the core distance, which is defined as the distance from a point to its k -th nearest neighbors, where $k = \text{min_samples}$.

Using these core distances, the mutual reachability distance between any two points x_i and x_j is defined as:

$$d_{\text{reach}}(x_i, x_j) = \max\{\text{core}(x_i), \text{core}(x_j), d(x_i, x_j)\}$$

This transformation adjusts pairwise distances to account for variations in local density. A weighted graph is then constructed using mutual reachability distances from which a minimum spanning (MST) tree is derived.

The MST is used to build a hierarchical clustering structure by progressively removing edges in order of increasing distance, resulting in clusters that correspond to different density levels. From this hierarchy, clusters are selected based on their stability which reflects how persistent a cluster remains across varying density thresholds.

Finally, data points belonging to stable clusters are assigned cluster labels, while points that do not belong to any cluster are labeled as noise.

Clustering Procedure

HDBSCAN clustering in this project is applied to four different representations of the data to evaluate the effect of dimensionality reduction:

- Raw embeddings (1024 dimension)
- PCA-reduced embeddings (50 dimensions)
- UMAP-reduced embeddings (10 dimensions)
- PCA + UMAP embeddings (50 -> dimensions)

For each representation, HDBSCAN is applied independently and the resulting cluster labels are stored for further analysis.

HDBSCAN Parameters

The following parameters are used for clustering in the project:

- `min_cluster_size = 3`
It defines the minimum number of points required to form a cluster. A small value is chosen here due to the limited dataset size (73 resumes), allowing smaller but meaningful groups to be identified.
- `min_samples = 1`
It controls how conservative the clustering is. Assigning lower values allows more points to be assigned to clusters rather than being labeled as noise.
- `metric = Euclidean`
It specifies the distance metric used for clustering. Euclidean distance is applied on the reduced feature set which is appropriate after dimensionality reduction.

Output and Visualization Methods

For each dataset representation, the clustering process produces:

- Cluster labels for each resume
- Number of clusters formed
- Number of noise points (labeled as -1)

The cluster labels are saved as *.npy* files for each configuration, enabling comparison across different dimensionality reduction approaches.

To analyze the clustering behavior, a 2-dimensional visualization is generated using UMAP. The reduced data is plotted, with each point colored according to its assigned cluster. The visualization provides an intuitive understanding of cluster separation, density of clusters and distribution of noise points.

Clustering is the final step of the pipeline, where semantically similar resumes are grouped together based on their embedding representations. The effectiveness of clustering depends on the quality of embeddings and dimensionality reduction, which are evaluated through comparison across different configurations.

6.6.Label Preparation for Evaluation

For external evaluation of clustering performance, a labeled dataset is constructed manually. An Excel file (*resume_groups.xlsx*) is created, where each resume is assigned a category label along with a brief justification based on its content.

Since filenames differ between the raw dataset and processed embeddings, a normalization step is applied to ensure filename matching. This includes removing filename extensions, special characters and formatting inconsistencies. The normalized filenames from the embedding pipeline are then matched with those in the labeled dataset. The resulting label array is stored and used for external evaluation of clustering performance.

6.7.Evaluation Metrics

Clustering performance is evaluated using internal and external metrics, along with semantic validation. These metrics provide complementary perspectives for assessing cluster quality, as no single metric is sufficient to fully capture the effectiveness of unsupervised clustering.

Internal Evaluation

Internal metrics such as Silhouette Score, Davies-Bouldin Index and Calinski-Harabasz Score are commonly used to evaluate cluster quality [2]. They assess cluster quality without relying on ground truth labels. These metrics evaluate how well data points are grouped within clusters and how distinct different clusters are from each other.

Silhouette Score:

The Silhouette Score measures how well a data point fits within its assigned cluster compared to other clusters. It considers both intra-cluster similarity and inter-cluster separation. It is calculated as:

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where:

- $a(i)$ = average distance between point i and all other points in the same cluster
- $b(i)$ = minimum average distance between point i and points in the nearest neighboring cluster

The silhouette score (S) ranges from -1 to 1. It is computed for each point, with the final score being the average over all points. The values closer to 1 indicates that the data points are well-clustered, values closer to 0 suggests overlapping clusters. If S value is lower than 0, the clusters are misclassified.

Davies Bouldin Index (DBI):

The Davies-Bouldin Index evaluates clustering quality by measuring the ratio of intra-cluster dispersion to inter-cluster separation. It is calculated as:

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{S_i + S_j}{M_{ij}} \right)$$

where:

- k = number of clusters
- S_i = average distance of points within cluster i to its centroid (cluster spread)
- M_{ij} = distance between centroids of cluster i and j

For each cluster, the worst-case similarity with other clusters is considered. A low DBI score indicates better clustering as it suggests clusters are compact and well-separated. This metric emphasizes minimizing overlap between clusters.

Calinski-Harabasz Score (CH):

The Calinski-Harabasz score evaluates clustering quality by comparing between-cluster variance to within-cluster variance.

$$CH = \frac{\text{between-cluster variance}}{\text{within-cluster variance}}$$

Mathematically,

$$CH = \frac{\text{trace}(B_k) / (k-1)}{\text{trace}(W_k) / (n-k)}$$

where,

- B_k = between-cluster scatter matrix
- W_k = within-cluster scatter matrix
- n = number of data points
- k = number of clusters

This metric favors clear separation and compact grouping. A higher CH value indicates better clustering. It means clusters are dense and well separated.

External Evaluation

External metrics like Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) evaluate clustering performance by comparing predicted cluster assignments with ground truth labels [2]. These metrics provide insight into how closely the clustering results align with manually defined categories.

The ground truth labels are used for evaluation are manually created based on resume content, as described in Section 6.6.

Adjusted Rand Index (ARI):

ARI measures the similarity between predicted clusters and true labels while adjusting for random chance. It is based on Rand Index (RI) which measures how similar two groupings are by calculating accuracy over all pairs of data points.

$$RI = \frac{TP+TN}{TP+TN+FP+FN}$$

where,

- TP : pairs correctly placed in same cluster
- TN : pairs correctly placed in different clusters
- FP : pairs incorrectly placed in same cluster
- FN : pairs incorrectly placed in different clusters

ARI is given as:

$$ARI = \frac{RI - Expected\ RI}{Max\ RI - Expected\ RI}$$

The Expected RI is computed using a probabilistic model based on cluster sizes, handled internally by *sklearn* library.

ARI evaluates how consistently pairs of data points are assigned to the same or different clusters in both partitions. The values close to 1 means very good alignment, 0 means random labeling and negative values suggest worse than random case. This metric is robust to chance and works well for comparing clustering outputs with known labels.

Normalized Mutual Information (NMI):

NMI measures the amount of shared information between cluster assignments and true labels.

$$NMI = \frac{2 \cdot I(X,Y)}{H(X)+H(Y)}$$

where:

- $I(X, Y)$ = mutual information between clustering X and labels Y
- $H(X), H(Y)$ = entropy of clusters and labels

The entropy $H(X), H(Y)$ are computed based in the distribution of data points across clusters and ground truth labels respectively. Mutual Information $I(X, Y)$ is calculated from the joint distribution of clusters and labels, measuring how much information is stored between them. They are derived from the contingency table and are computed using *sklearn* library.

The NMI quantifies how much knowing the cluster assignment reduces uncertainty about the true label. The value 1 means perfect agreement and 0 means there is no mutual information.

Semantic Validation

In addition to standard metrics, semantic validation is performed using cosine similarity in the embedding space. The average similarity between resumes within the same cluster is compared with similarity between different clusters.

Let,

- sim_{intra} = average similarity within clusters

- $\text{sim}_{\text{inter}}$ = average similarity between clusters

A good clustering result satisfies :

$$\text{sim}_{\text{intra}} > \text{sim}_{\text{inter}}$$

This indicates that resumes grouped within the same cluster are more semantically similar to each other than to those in other clusters.

7. Results and Discussion

7.1. Embedding Validation Output

Before performing clustering, the quality of the generated embeddings is validated to ensure that they provide a meaningful representation of resume data.

Embedding Size:

The embedding matrix has a shape of (71,1024) where each resume is represented as a 1024-dimensional vector.

Normalization Check:

The L2 norms of the embeddings are analyzed to verify normalization:

Min norm	0.9999998
Max norm	1.0000001
Mean norm	1.0

These values confirm that the embeddings are properly normalized, ensuring that cosine similarity is a reliable measure of semantic similarity.

Cosine Similarity Distribution:

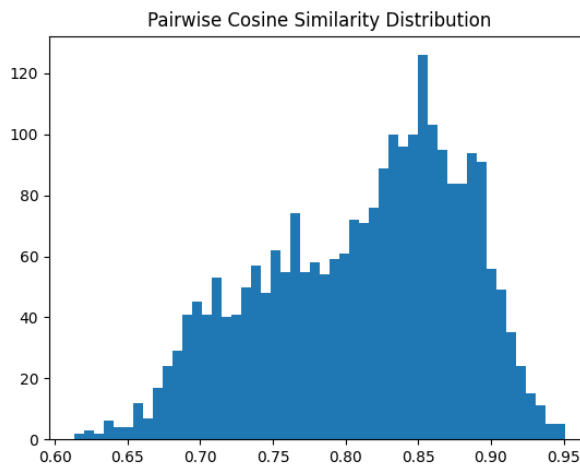


Figure 7.1: Pairwise cosine similarity distribution of resume embeddings

The cosine similarity values range from 0.61 to 0.95 with an average of 0.81. The distribution shows a widespread rather than a sharp peak, which indicates meaningful semantic validation across resumes.

However, the relatively high average similarity suggests that resumes share many common content, which contributes to challenges in cluster separation observed in later stages.

Nearest Neighbor Analysis:

A Nearest neighbor analysis is done to verify that similar resumes are located close to each other in embedding space, like academic profiles grouped together with other academic resumes, and so on.

The outputs were manually screened for verification. A few samples for validation are:

Resume 1: Julia_Kempe.txt

Nearest Neighbors	Similarity	Reason for similarity (Manual Verification)
resume.txt	0.934	Similar academic CV research, publications and teaching sections
Flavio_Calmon_CV.txt	0.927	Professor-level academic CV with similar document structure and research-oriented content
purdue.txt	0.927	Academic faculty profile with research-focused content and comparable CV format
CV-Mahdi-Cheraghchi.txt	0.927	Research-oriented academic CV with similar structure and publication content
Tanya Berger-Wolf.txt	0.921	Senior academic CV with structured sections and research-focused contents

7.2. Clustering Performance Overview

The embedding generation strategy plays a crucial role in the overall performance of the clustering pipeline. The use of instruction-based embeddings helps guide the model toward capturing relevant aspects of the resume, such as career stage and role. Anchor based filtering further refines the representation by selecting only the most relevant chunks of text. Weighted aggregation ensures that more important sections contribute more to the final embedding.

This combination of techniques improves the quality of the embeddings by reducing noise and emphasizing meaningful information. As a result, the clustering algorithm operates on more informative representations, leading to better cluster formation.

Clustering performance is evaluated across four representations of the data: Raw embeddings, PCA-reduced embeddings, UMAP-reduced embeddings and PCA followed by UMAP. The resulting scatter plots are as follows:

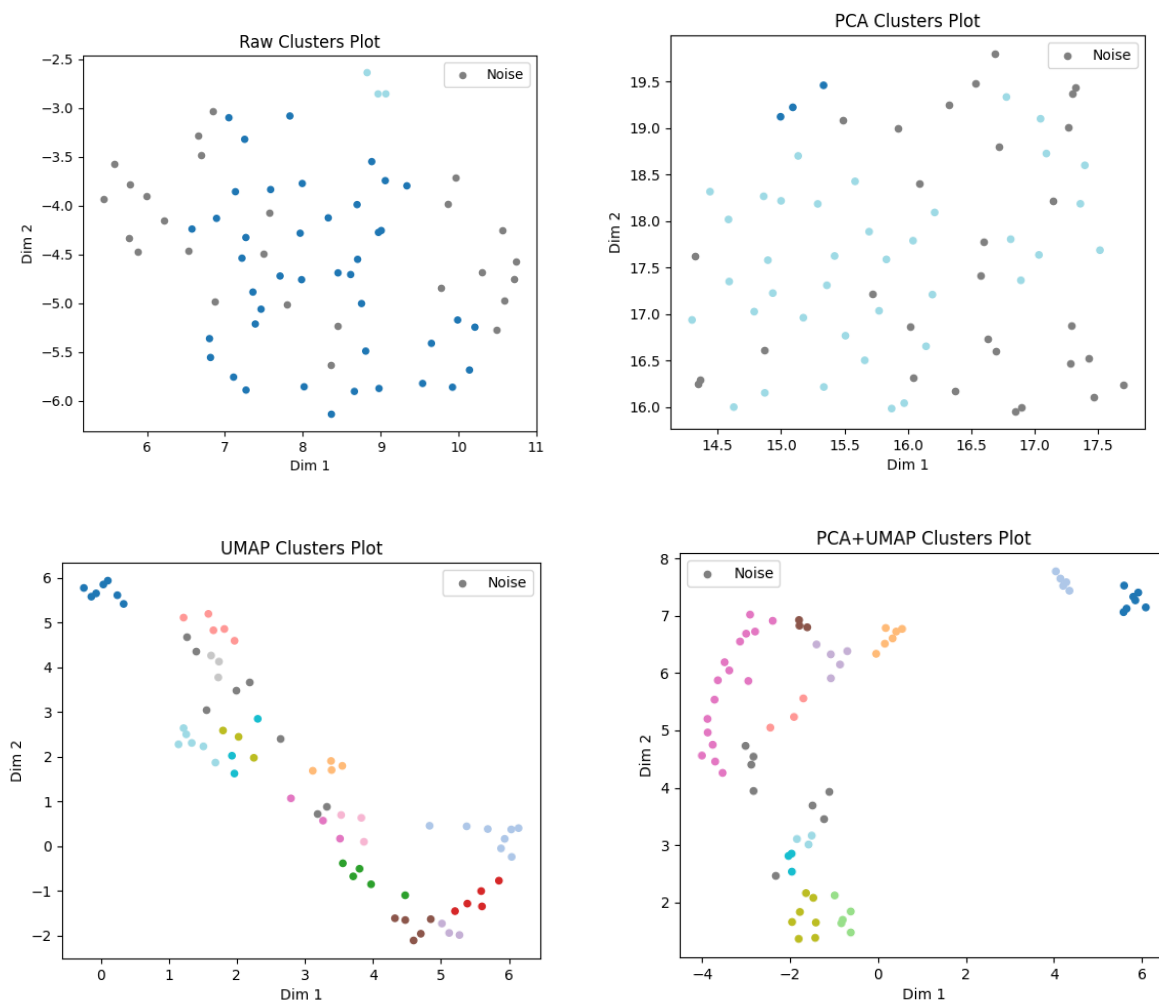


Figure 7.2.1 Clustering results across different representations

The final cluster assignments for PCA+UMAP configuration is represented in table below:

Cluster	No. of resumes	Content
Noise	4	Chakraborty_CV2025.txt Subbarao Kambhampati.txt Flavio_Calmon_CV.txt texas-tech.txt
Cluster 0	7	DiekhoffLadelle _ Ball State University.txt Jennifer Coy _ Ball State University.txt Travis Faas _ Ball State University.txt Benjamin Diekhoff _ Ball State University.txt Huseyin Ergin _ Ball State University.txt Kunwarjay Bagga _ Ball State University.txt Andrew Harris _ Ball State University.txt
Cluster 1	5	Umit Ogras _ ASU Search.txt Jennifer Kitchen _ ASU Search.txt Sandeep Gupta _ ASU Search.txt Hugh Barnaby _ ASU Search.txt Mariana Bertoni _ ASU Search.txt
Cluster 2	6	Luis_von_Ahn.txt nikolabanovic_cv.txt Arthur_CV.txt Paul Gestwicki _ Ball State University.txt ScottDexterCV.txt Amos Bloomberg.txt
Cluster 3	5	CV - Adeline Paiement.txt Daniele Panozzo.txt Denis Zorin.txt David Fouhey.txt CV-Jun Gao.txt
Cluster 4	3	Abhishek Gupta.txt Resume_main.txt Zak Kassas.txt
Cluster 5	5	Dartmouth.txt Debdipta Goswami.txt CV_TeYenWu.txt Robert Gregg CV - rdgregg-CV_web.txt penn.txt
Cluster 6	3	Roberto Myers.txt CurriculumVitae.txt buffalo.txt
Cluster 7	16	mosharaf_cv.txt hviettung_CV.txt vitae.txt CV_MikeVDBurmester.txt CV-Mahdi-Cheraghchi.txt

		Nir Bitansky.txt Ohio-State.txt Mohamed Zahran.txt Resume - Jiasi Chen - JiasiChen_CV.txt Carole-Jean Wu.txt cv-rachit.txt 24-11-21-CV.txt CV_GrigoryFedyukovich.txt cv (2).txt cv_abdelfattah.txt cv_An-IAWng.txt
Cluster 8	4	Philip Schniter.txt SWicker_CV.txt Lee Potter.txt Aylin Yener.txt
Cluster 9	7	Julia Kempe.txt Harvard-cv.txt resume.txt Joan Bruna.txt LAURA BALZANO - Balzano_CV_June_2020.txt Ohio.txt Shayok Chakraborty CV.txt
Cluster 10	3	purdue.txt CV_XianFMallory.txt MG_CV_2026_02.txt
Cluster 11	3	Academic_CV.txt cv.txt Tanya Berger-Wolf.txt

The number of clusters and noise points produced by each method varies significantly.

- Raw embeddings produce only 2 clusters with a large number of noise points, indicating weak structure in the original high-dimensional space.
- PCA-reduced data shows similar pattern, with only 2 clusters and a slightly higher number of noise points.
- UMAP-reduced data results in 14 clusters with 8 noise points, indicating improved separation and more meaningful grouping.
- PCA followed by UMAP produces 12 clusters with the lowest number of noise points i.e.4 among all methods, suggesting a more stable clustering structure.

The clustering results indicate that resumes are grouped based on semantic similarity rather than strict categorical boundaries. Internal evaluation metrics show that clusters formed after dimensionality reduction are more compact and better separated.

For example, resumes from the same institution (e.g. Ball State University or ASU) are frequently grouped together, indicating that the model captures contextual and institutional similarities.

However, perfect separation is not observed. This is expected due to the inherent nature of resume data. Many resumes, especially in technical fields, share common skills such as programming languages, framework and tools. This overlap leads to high similarity between different categories, making it difficult to form clearly distinct clusters.

HDBSCAN is particularly suitable in this context because it does not impose rigid cluster boundaries. Instead, it identifies dense regions in the data and allows for flexible cluster shapes [5]. Additionally, its ability to label certain points as noise is useful for handling resumes that do not strongly belong to any specific group.

To further validate clustering results, sample resumes from the same cluster are visually inspected. These resumes show similar structure, content organization and professional characteristics such as academic roles and research focus.

Example snapshots from selected clusters are included to illustrate these similarities.

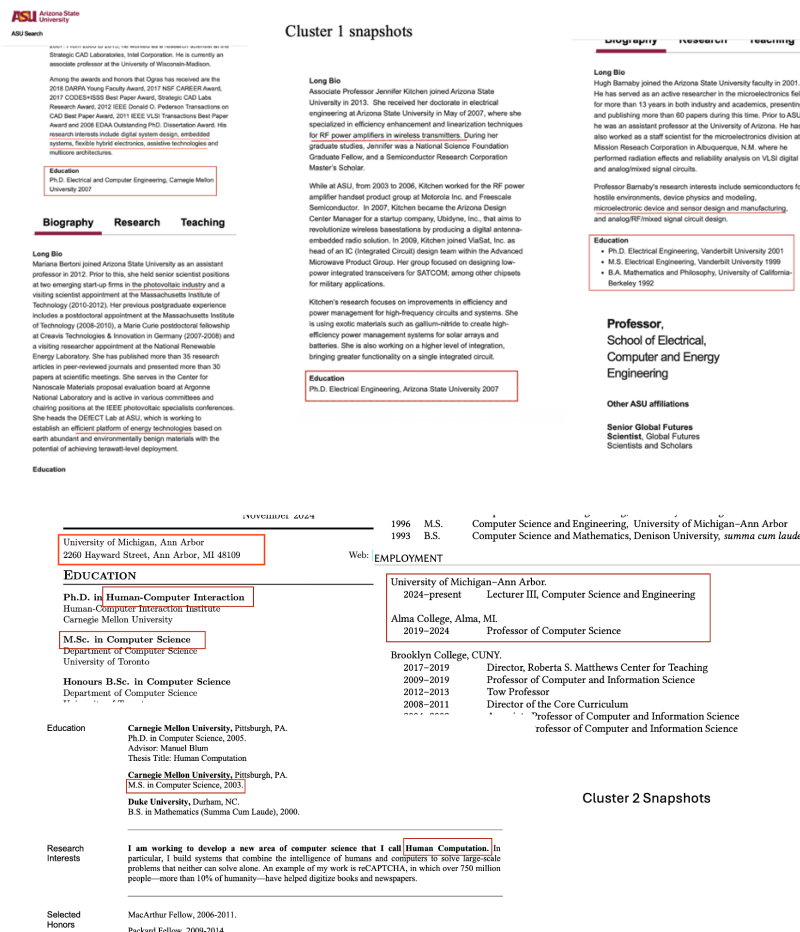


Figure 7.2.2. Snapshots showcasing important keywords in different resumes from a single cluster

7.3. Effect of Dimensionality Reduction

A clear improvement in clustering quality is observed after applying dimensionality reduction.

- Raw embeddings show low silhouette scores and high Davies-Bouldin values, indicating poor cluster separation.
- PCA improves these metrics slightly, suggesting that reducing dimensionality helps remove some noise,
- UMAP significantly improves all evaluation metrics, indicating better separation and compactness of clusters.
- The PCA+UMAP configuration achieves the best overall performance, with the highest silhouette score and Calinski-Harabasz score, and the lowest Davies-Bouldin index.

The results demonstrate that dimensionality reduction plays a critical role in improving clustering performance. Raw embeddings produced poor clustering results, characterized by low silhouette scores and high Davies-Bouldin index values. This indicates that high-dimensional embedding spaces contain noise and redundant information that negatively affect clustering.

Applying PCA resulted in a slight improvement, suggesting that linear dimensionality reduction helps remove noise and compress the representation. However, PCA alone is insufficient, as it cannot capture nonlinear relationships between data points.

In contrast, UMAP significantly improves clustering performance across all metrics. This aligns with prior work for showing that non-linear dimensionality reduction techniques are more effective for clustering high-dimensional data, as they preserve both local and global structure in the data space [5]. UMAP enables better separation of semantically distinct resumes by projecting them into a lower-dimensional space where clusters are more distinguishable.

The combination of PCA and UMAP achieves the best overall performance. This suggests that applying PCA before UMAP helps remove noise and stabilize the input representation, allowing UMAP to more effectively learn meaningful structure. This sequential approach is commonly used in practice for improving clustering outcomes, especially when dealing high-dimensional embeddings.

7.4. Evaluation Results

Internal metrics such as Silhouette Score, Davies-Bouldin Index and Calinski-Harabasz Score are selected because they evaluate cluster compactness and separation without relying on ground truth labels. These metrics are widely used in clustering evaluation and are particularly useful when labels are unavailable or unreliable [2].

The results are summarized in table below.

Method	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
Raw	0.0989	1.7155	2.6768
PCA	0.1422	1.5089	3.3183
UMAP	0.3913	0.7750	43.5688
PCA + UMAP	0.3944	0.7343	56.2254

The internal evaluation scores show clear improvement in clustering quality across different representations.

External metrics such as Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) are used to compare clustering results with ground truth labels. These metrics provide insight into how well the clustering aligns with predefined categories. ARI measures agreement between clustering and labels while adjusting for chance, whereas NMI measures shared information between the two assignments.

Metric	Value
Adjusted Rand Index (ARI)	0.0436
Normalized Mutual Information (NMI)	0.3245

The external results show a low ARI and moderate NMI, indicating limited agreement between predicted clusters and ground truth labels.

This does not necessarily imply poor clustering performance. Instead, it reflects a mismatch between semantic similarity and predefined labeling. The ground truth labels are manually assigned and may not fully capture the underlying semantic structure of the data.

Resume data often contains overlapping categories. For example, a candidate may have both academic and industry experience or a combination of research and practical skills. Such resumes may be semantically closer to multiple groups, making strict categorical labeling difficult.

Clustering, on the other hand, groups data based on similarity in the embedding space. As a result, clusters may capture nuanced relationships that are not reflected in the labeling scheme. This explains why clusters contain mixed labels and why ARI is low.

The moderate NMI value suggests that while there is some shared structure between clusters and labels, the alignment is not strong. This is expected in real-world datasets where labels are not perfectly separable.

Importantly, prior work has highlighted that external evaluation metrics such as ARI and NMI rely heavily on the quality and correctness of ground truth labels. When labels are noisy, overlapping or do not reflect true data distribution, these metrics may not accurately represent clustering quality [2]. Therefore, low ARI and NMI values in this context should be interpreted cautiously and considered alongside internal and semantic evaluation results.

7.5. Semantic Validation Results

Semantic Validation is conducted by comparing cosine similarity within clusters and between clusters.

Metric	Value
Within-cluster similarity	0.8601
Between-cluster similarity	0.8023

Semantic validation results show that the average cosine similarity within clusters is higher than between clusters. This indicates that resumes grouped together are more semantically similar, confirming that the clustering process captures meaningful relationships.

However, the difference between within-cluster and between-cluster similarity is relatively small. This suggests that the dataset has a high degree of overall similarity, which is typical for technical resumes that share common terminology and skills.

This observation further explains the difficulty in achieving strong separation and external evaluation scores.

7.6. Summary of Observations

- Clustering performance improves significantly after applying dimensionality reduction.
- UMAP plays a major role in improving cluster separation.
- The PCA+UMAP configuration produces the most stable and well-defined clusters.
- External evaluation shows limited alignment with ground truth labels.
- Semantic validation confirms that clusters capture meaningful similarity, although overall dataset similarity remains high.

7.7. Limitations

- The dataset size is relatively small, which may limit the generalizability of the results.
- Resume data inherently contains overlapping categories, making clear separation difficult.
- Ground truth labels may not accurately reflect the underlying semantic structure.
- Some resumes are identified as noise by HDBSCAN, indicating that not all data points can be confidently assigned to clusters.

8. Conclusion

This study presents an unsupervised approach for clustering resume data using transformer-based embeddings, dimensionality reduction techniques and density-based clustering. The results demonstrate that directly clustering high-dimensional embeddings is ineffective due to the dense and overlapping nature of resume data. However, applying dimensionality reduction significantly improves clustering performance.

Among the evaluated methods, UMAP provides the most substantial improvement by preserving local structure in the data, while combination of PCA and UMAP yields the best overall performance across all internal evaluation metrics. This indicates that combining linear and non-linear dimensionality reduction techniques helps in capturing both global and local relationships within the data.

External evaluation using ARI and NMI shows limited alignment between predicted clusters and manually defined labels. This is primarily due to the inherent ambiguity and overlap in resume categories, as well as the difference between semantic similarity and label-based grouping. Despite this, semantic validation confirms that resumes within clusters exhibit higher similarity than those across clusters, indicating that meaningful grouping is achieved.

Qualitative analysis of cluster outputs further supports these findings, showing that resumes are grouped based on shared characteristics such as academic roles, research focus and institutional affiliation.

Overall, the study highlights the challenges of clustering highly similar and semi-structured text data, while demonstrating that meaningful structure can still be extracted using appropriate preprocessing and dimensionality reduction techniques.

Future work may explore incorporating structured features such as skills, experience levels or domain-specific keywords to improve cluster inseparability as well as evaluating alternative embedding models and clustering approaches.

REFERENCES

- [1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," arXiv:1908.10084, 2019.
- [2] J. Min, Y. Zhang, and others, "A Comprehensive Survey on Deep Clustering: Taxonomy, Challenges, and Future Directions," arXiv, 2022.
- [3] BAAI, "BGE Large English Embedding Model," Hugging Face, 2023. [Online]. Available: <https://huggingface.co/BAAI/bge-large-en-v1.5>
- [4] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [5] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," arXiv:1802.03426, 2018.
- [6] L. McInnes, J. Healy, and S. Astels, "HDBSCAN: Hierarchical Density-Based Clustering," Journal of Open Source Software, 2017.
- [7] I. T. Jolliffe, "Principal Component Analysis," Springer, 2002.
- [8] M. Grootendorst, "BERTopic: Neural topic modeling with a class-based TF-IDF procedure," arXiv:2203.05794, 2022.
- [9] "Analyzing the Effects of K-Means and Random Forest Algorithms on Resume Classification through Manipulation of Cluster Sizes and Datasets," IEEE, 2024.
- [10] "DeepSkillNER: Resume Screening using Deep Learning and Spectral Clustering," Multimedia Tools and Applications, Springer, 2023.